Falsifying Authorization Interfaces for Tool-Using Agents: Counterfactual Validation of Policy-Relevant Effects

Anonymous Authors

Abstract

Authorization logic can enforce only distinctions visible through its observation interface. If that interface maps two policy-distinct executions to the same representation, every downstream authorizer faces the same choice: admit an unauthorized execution or withhold authorized work. Tool-using agents make this interface difficult to construct because defaults, aliases, list expansion, and pre-state can make a structured call commit effects that its name and explicit arguments do not reveal.

We turn authorization observability into an executable security contract. Registration treats a candidate tool descriptor as an untrusted hypothesis, executes controlled call and state interventions in copied sandboxes, observes committed transitions, and asks a policy-separation oracle whether the transition difference requires another decision. A policy-separating pair that the candidate merges is a concrete failure certificate. Counterexamples drive refinement; the resulting validation record binds the frozen descriptor to the implementation, intervention manifest, and policy family used for validation.

Source execution produces failure certificates for tool-name, raw-call, and common-field interfaces. We instantiate the method with a typed-effect candidate and test 11 tools from three public Model Context Protocol (MCP) implementations. Across 264 post-freeze contexts, the coarse interfaces retain policy-mixed cells, while the typed candidate and a strong state-aware request interface preserve every tested distinction. A native-delta policy reaches the same decisions as the typed path, and systematic descriptor mutations test registration's sensitivity to concrete faults. In a 1,024-context authority protocol, two independently constructed interfaces—typed effects and a strong state-aware request—both support exact decisions from one shared ACL, capability, and delegation engine, showing that validation is representation-agnostic. Finally, a pre-commit case study demonstrates how a frozen interface can be consumed without placing authority in the LLM. The result is a falsification-first method for auditing any pre-commit authorization interface before policy trusts it.

1 Introduction

A user asks an agent to create a meeting with Alice. After reading an untrusted document, the agent calls the calendar tool with both Alice and Eve as participants. The call creates the requested event and also commits an attacker-chosen invitation, a common indirect-prompt-injection failure in tool-using agents [8,28]. The authorization question is not whether the agent may invoke `create_event`. It is whether this execution may create this event, invite Alice, invite Eve, and apply the chosen visibility.

Authorization cannot enforce a distinction that its observation interface has already erased. An ACL may permit Alice and reject Eve, but a monitor that sees only the tool name cannot apply that rule. Provenance may identify where the string "Eve" originated without identifying whether it controls an invitation, a message, or an inert note. Explicit arguments expose more of the request, yet defaults, aliases, list expansion, and pre-state can change the committed effect without changing the call text. Downstream model reasoning is subject to the same information boundary.

Figure 1 illustrates this boundary. Let ρ be the execution representation available to an authorizer. If $\rho(u) = \rho(v)$ while an admissible policy must decide u and v differently, every deterministic consumer of ρ returns the same decision for both. Allowing admits one unauthorized execution; denying or abstaining withholds one authorized execution. Complete mediation and least privilege therefore require an adequate object of mediation. We call this representation obligation *tool-effect binding*.

Agent harnesses make tool-effect binding a first-class systems problem. They register evolving third-party tools from schemas and natural-language descriptions, ask an untrusted model to synthesize calls, and resolve important semantics inside the execution layer. A call may expand one list into several recipients, choose a default mode, overwrite an existing resource, or create a different transition under another pre-state. Unlike a security-typed API, the harness is rarely given a trusted effect object that already captures these distinctions.

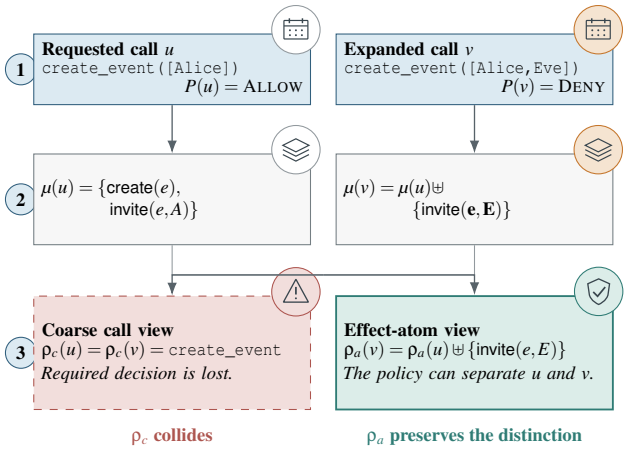

Figure 1: An executable authorization-separating counterexample. Calls u and v use the same tool, but source execution shows that v adds an invitation that the policy rejects. Their collision falsifies the coarse view for every domain containing this pair. The typed-effect interface preserves the distinction and can therefore expose it to policy.

A model can propose such an object, but the proposal supplies neither semantic evidence nor authority.

We test proposed authorization interfaces against the implementations they will mediate. The evidence is asymmetric: one executable pair with different policy decisions and an identical representation falsifies the interface wherever the pair is reachable. Successful registration binds the frozen descriptor to its implementation, interventions, policy family, and unresolved tests, producing an auditable contract for downstream use.

Registration treats a candidate descriptor as an untrusted hypothesis. It executes a valid call and controlled variants in copied sandboxes, varying fields, omissions, list cardinality, defaults, provenance, or pre-state. A source oracle records each committed transition, and a policy-separation oracle identifies differences that require another decision. A collision yields an executable failure certificate and forces refinement. Invalid and unresolved interventions remain in the validation record.

We instantiate this method with one candidate representation, typed effect atoms. Each atom represents one policy-relevant effect occurrence and retains the resource, target, operation, provenance, or qualifier needed by the policy family used during validation. List-valued calls expand when their members require independent decisions. The validation criterion applies to any candidate interface, so the framework is representation-agnostic. We study two designs: typed effects place tool-specific semantic adaptation in a registered descriptor, while a strong state-aware request places it in a downstream adapter. Their comparison measures where each design places trusted state, code, normalization, and runtime cost.

The evaluation follows this chain from implementation behavior to authorization. Source replay establishes compound and state-dependent effects; executable pairs expose collisions in coarse interfaces; and post-freeze tests exercise 11 tools from three public MCP implementations. A four-domain benchmark isolates representation choice under fixed calls, state, authority, and decision logic. Copied-sandbox and AgentDojo studies then consume frozen interfaces at audited pre-commit boundaries.

This paper makes three contributions:

- **Authorization observability.** We formalize the upper bound that an observation interface places on every downstream authorizer and derive the unavoidable safety–availability consequence of a policy-separating collision.
- **Executable interface falsification.** We combine source-executed effect witnesses, policy separation, and representation collisions into concrete failure certificates and a counterexample-guided registration discipline for existing tools.
- **Validated interfaces and consumption.** We falsify coarse views on project-local and independently implemented tools, evaluate typed-effect and state-aware interfaces under shared ACL, capability, and delegation authority, quantify their semantic-placement tradeoffs, and exercise a frozen descriptor at pre-commit integration points.

An authorization interface is therefore an implementation-grounded contract, not a declaration to be trusted on sight.

2 Related Work

Agent defenses reason about several interfaces around a tool call: why the planner selected it, where its inputs originated, what the implementation may do, and whether the represented action is authorized. We focus on the missing validation step between execution semantics and authorization: whether the object exposed to policy preserves the implementation-level distinctions that policy needs.

Action selection and causal attribution. AgentDojo, ToolEmu, and AgentLAB expose indirect prompt injection in tool-rich and long-horizon tasks [8, 19, 28]. Prompt defenses and step classifiers assess whether a proposed action appears safe [17, 25]; Spotlighting delimits tool output and PromptArmor sanitizes untrusted content [16, 32]. CaMeL and IPIGuard preserve trusted control or track data dependencies [1, 7]; AttriGuard and CausalArmor intervene on observations and rerun the agent to estimate which input caused a proposed call [15, 20]. These systems attribute an action to its upstream causes. Our interventions operate at the next boundary: they vary the call or pre-state, execute the tool, and

test whether the authorization interface preserves the resulting policy distinction.

Authority, provenance, and mediation. Agent policy systems restrict tools, actions, parameters, or capabilities [23, 31, 37–39]. PACT assigns semantic roles to arguments, while AuthGraph relates clean authority to argument provenance [12, 36]. SecureClaw and commit-time authorization place fresh checks at the effect sink [24, 30]. These systems supply authority or enforcement mechanisms. Their precision depends on the object presented for authorization: provenance identifies origin, call arguments encode requests, and policy decides represented effects, but none of these components alone establishes that the representation matches what the implementation commits. Our enforcement model builds on complete mediation, least privilege, confused-deputy, information-flow, and provenance principles [4, 5, 10, 11, 14, 29]. Contextual security models likewise separate task, action, source, and data boundaries [34].

Industrial agent harnesses. DeepSeek Harness routes tool calls through a plugin-based pre-execution waterfall with allow, deny, and ask decisions, monotonic guards, one-shot approval, and an immutable logged result [9]. Its Cordis substrate formalizes revertible effects and reactive coeffects for dynamic composition [33]. These systems provide an enforcement pipeline but do not validate whether the representation exposed to that pipeline preserves the policy-relevant committed effects of a tool call; our registration method targets exactly that input.

Effect specification. Type-and-effect systems make effects explicit by construction [22], and ETAS brings effect rows and compiled monitors to an agent language [35]. Contract2Tool infers planning contracts, ContractGuard protects supplied contracts, and ToolGuardian combines descriptions, traces, mock execution, and source analysis to vet tools [2, 18, 27]. These techniques produce, protect, or analyze semantic descriptions. We treat such a description as a candidate authorization interface and ask whether its induced equivalence classes preserve the distinctions required by policy.

Counterexample-guided interface validation. Counterexample-guided abstraction refinement splits a coarse abstraction after concrete execution falsifies it [6]. Causal abstraction asks whether interventions at a lower level are preserved by a higher-level model [3, 13, 26]. We apply the same executable discipline to an authorization boundary. A source intervention establishes a semantic difference, a policy oracle establishes its security relevance, and equality under the candidate interface completes a failure certificate. The resulting validation record makes authorization observability policy-relative, implementation-grounded, and testable.

3 Problem and Security Model

System and enforcement boundary. A user gives an agent an authenticated task q. The agent may read messages, documents, webpages, or database records before proposing a structured tool call $c = (t, \bar{x})$. The large language model (LLM) is an untrusted control-plane component: its plan, effect description, and arguments are proposals rather than authority. Executing the totalized call from pre-state s may commit several externally visible effects, such as creating an event, inviting principals, or changing visibility. Enforcement occurs after argument totalization and immediately before commit. The executor may commit only the exact call associated with the monitor's decision.

Threat model. The adversary controls content returned by one or more untrusted reads. It may embed instructions or attacker-chosen values, induce the agent to add a resource, target, payload, operation, or effect, and carry this influence across a multi-step trajectory. We assume the agent may follow the injected instruction completely; security does not rely on model alignment. The adversary cannot modify the authenticated task, forge trusted provenance, change a registered descriptor or policy, compromise the monitor, or replace a checked call before execution. It also cannot replace the registered tool implementation without triggering revalidation.

Representation and commit objective. Let $u = (t, \bar{x}, s, h)$ denote an execution context containing a tool, totalized arguments, trusted pre-state, and provenance evidence. Let $\mu(u)$ denote its committed effects. A policy context P may additionally depend on trusted execution metadata $\gamma(u)$, including effect provenance. Writing $\omega(u) = (\mu(u), \gamma(u))$, the policy context P induces an ideal per-commit predicate $I_P(\omega(u))$. The desired commit condition is $I_P(\omega(u)) = 1$.

The explicit-authority experiment instantiates P with authenticated subject identity, resource ACLs, held capabilities, and attenuated delegations. The earlier source-relation diagnostic uses effect-set containment only as a finite interface-discrimination stress relation. In both settings, the descriptor is an observation contract rather than a source of authority: the authenticated user, harness, or external policy component supplies P, and the planner cannot enlarge it. A separate AgentDojo integration implements a narrower provenance-origin predicate. It tests mediation and auditability, not organization-wide entitlement.

Registration-time trust assumptions. Offline validation assumes that the copied sandbox faithfully runs the versioned implementation, the source execution oracle observes the exercised committed transition, interventions are reproducible, and the policy-separation relation is supplied independently of the candidate descriptor. These privileged observations

generate registration evidence and do not sit on the deployed commit path. Failed or unresolved executions remain explicit and cannot justify dropping a field.

Runtime enforcement TCB. Runtime trusts the descriptor–implementation binding, authority and policy state, the provenance adapter where used, the monitor, and the exact-call executor. State-dependent instantiation additionally requires a fresh trusted pre-commit snapshot. The conditional guarantee assumes complete mediation and check–use consistency for both the call and policy snapshot. If the planner can alter authority, relabel provenance, replace the checked call, or race the trusted state, it has crossed the stated boundary. Section 8 discusses semantic drift, concurrency, remote effects, and output-only goals.

4 Executable Authorization-Interface Validation

Figure 2 separates offline validation from downstream consumption. Registration tests an untrusted candidate descriptor against the tool implementation; the frozen descriptor then supplies an observation interface to authenticated authority. A developer or LLM may seed names and bindings. Registration supplies the semantic evidence, and the policy supplies authority.

An *effect atom* is one candidate policy-relevant unit. A *tool descriptor* is the executable, security-critical observation contract produced by offline registration. Its instantiated atoms expose the distinctions needed by the policy family used during validation.

Registration first establishes an *effect witness*: a committed-state difference showing that an intervention changes what the tool does. An *authorization-separating witness* further establishes that an admissible policy decides the resulting semantic views differently. Their collision under the candidate representation falsifies that interface on every domain containing the pair. The validation record binds an accepted descriptor to its implementation, intervention domain, policy family, and unresolved tests.

The three predicates compose into one executable failure certificate when

$$\omega(u) \neq \omega(v),$$
$$I_P(\omega(u)) \neq I_P(\omega(v)), \qquad \rho(u) = \rho(v).$$

We call this triple a *failure certificate*.

4.1 From Committed Effects to Authorization Observations

Let $u = (t, \bar{x}, s, h)$ contain a tool name, totalized arguments, pre-state, and typed provenance evidence, and let $\mu(u)$ be the multiset of committed state changes produced by executing u. As in Section 3, $\omega(u)$ augments this multiset with policy-relevant trusted execution metadata. An *effect atom* records one occurrence that an admissible policy may need to authorize independently. For example, adding a participant to an existing calendar event can produce

$$\langle \mathsf{invite}, \mathsf{event{:}17}, \mathsf{sarah@example.com}, \mathsf{commit}, \mathsf{tool\text{-}output} \rangle.$$

Here the components denote effect kind, operated resource, target principal, operation or commit mode, and control provenance. A tool-specific atom may also retain a payload, visibility, date, or other qualifier when the policy family can distinguish its values. Lists expand per target or resource when their members can be authorized independently.

A candidate typed-effect descriptor d_t contains (i) argument-totalization rules, (ii) proposed effect kinds, (iii) effect-bearing schema fields and their bindings or roles, (iv) inactive values, and (v) counterfactual evidence for those choices. Its instantiation function defines the monitor view

$$\rho_d(u) = A_{d_t}(u) = \text{instantiate}(d_t, \bar{x}, s, h),$$

a canonical multiset of concrete atoms. Thus $\mu(u)$ records what execution commits, while $\rho_d(u)$ is the representation available before commitment. The validation criterion applies to any representation ρ.

Atom equality is field-wise after descriptor-defined canonicalization, and multiplicity is preserved: inviting the same principal to two resources is not collapsed into one occurrence. A general descriptor may bind aliases to stable resource or principal identifiers. The evaluated descriptor canonicalizes case and whitespace, numeric and date forms, normalized URLs, and values linked by a matching authorized-read ledger entry. Alias-sensitive and typed-resource views are evaluated separately.

Policy-relative minimality. Minimality is relative to a frozen intervention domain D and policy family $\mathcal{P}$. A representation is minimal on $(D, \mathcal{P})$ when merging any two of its classes creates a pair that some $P \in \mathcal{P}$ decides differently. Accordingly, changing committed state establishes that a field is effect-relevant. A trusted provenance or control-source change can also be authorization-relevant without changing state. Either establishes authorization necessity when an admissible policy separates the resulting semantic views. A returned-string change alone establishes neither. Minimality describes the induced partition on $(D, \mathcal{P})$.

4.2 Falsification-First Registration

Registration consumes a versioned tool schema and implementation, sandbox states, an intervention catalog, and, when authorization sufficiency is being tested, a policy-separation oracle from $\mathcal{P}$. It proceeds as follows.

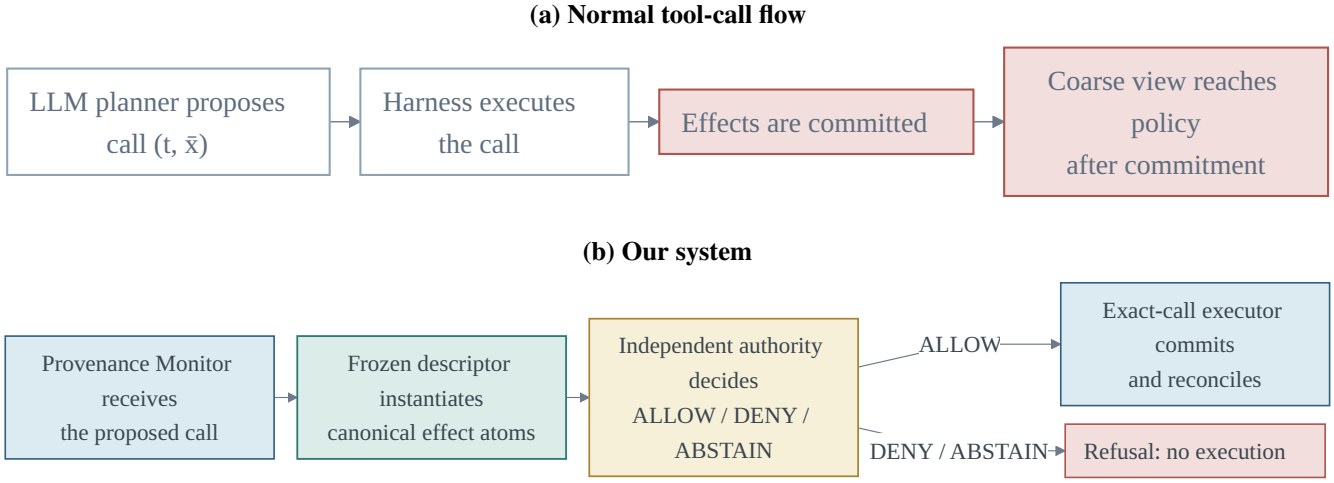

Figure 2: A normal harness commits a proposed call and exposes only a coarse post-commit view to policy. Our system mediates the same call before commit: a deterministic monitor instantiates a frozen descriptor into canonical effect atoms, consults independent authority, and the exact-call executor commits only an allowed call, whose decision and execution are then reconciled.

1. Propose. A developer or an LLM proposes effect names and field bindings from the tool name, description, and schema. Registration treats this proposal as an untrusted initial hypothesis. Executable evidence determines which bindings enter the frozen descriptor.

2. Intervene and execute. For a valid base call, the harness changes a field value or its presence while holding the implementation and initial state fixed. Interventions include typed alternatives, observed alternatives, boundary values, list expansion, and omission/default cases when supported. Surface-only variants serve as placebos. Both calls execute in copied sandboxes, so no external side effect is produced.

The source execution oracle observes post-execution state in a copied sandbox and answers *what happened?* The policy-separation oracle asks *does this semantic difference require another decision?* The descriptor answers a third question: *what can policy observe before commit?* Registration uses post-execution state as offline evidence; runtime authorization uses the validated pre-commit view.

The third-party validation adds a native-delta decision path to check the typed path through a different semantic interface. Section 6 describes the isolation between them.

3. Classify evidence. A state-difference oracle compares committed state, returned output, and execution validity; provenance interventions additionally use the trusted harness annotation. The harness records *committed-effect changed*, *policy-metadata changed*, *output-only changed*, *semantic-invariant*, or *invalid/unresolved*. The first is evidence that a field participates in the exercised effect relation; the second records a change such as trusted control provenance. A policy-separating decision over either class completes a failure certificate when the candidate representation merges the two semantic views. An invariant outcome supports dropping the field for the recorded states and interventions. Invalid outcomes remain unresolved and provide no evidence for omission.

4. Refine, retain, or freeze. A developer, reviewer, or tool-specific refinement rule uses a failure certificate to add a field, split a list-valued effect, or introduce a qualifier. Repeating this step implements the refinement procedure analyzed in Section 5. A descriptor is accepted when its recorded test domain and policy family contain no remaining separating collision. The AgentDojo registry conservatively retains fields with effect-change evidence or unresolved tests; a later multi-base audit broadens this evidence while preserving the frozen registry. The descriptor, implementation identity, intervention manifest, and evidence are hash-frozen together. Runtime loads this frozen record. Figure 3 summarizes the loop.

Running example. Consider a calendar tool whose participant argument is a list. The proposal may initially describe the call only as create-event. Registration executes a base call that invites Alice and a paired call that additionally invites Eve. If the state oracle observes a second invitation and the policy allows Alice but not Eve, the pair is a failure certificate. Refinement therefore expands the participant list into one invite atom per principal. A surface paraphrase that preserves the same event and participants should leave the atom multiset unchanged. The frozen record binds the descriptor to the calendar implementation, interventions, and policy family exercised by these checks.

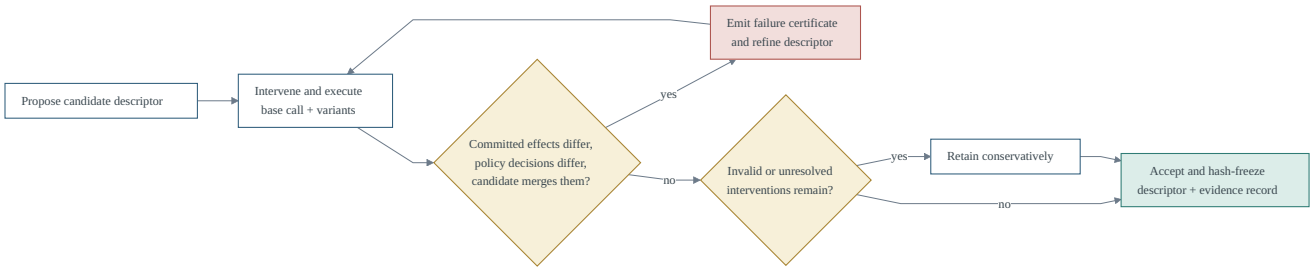

Figure 3: Falsification-first registration loop. Each collision yields an executable failure certificate and a refinement step; descriptors are hash-frozen only after the tested domain and policy family contain no remaining separating collision.

4.3 Downstream Consumer Contract

For context $u = (t, \bar{x}, s, h)$, the pre-commit consumer applies the descriptor's totalization rules and instantiates $\rho_d(u)$ as defined above. Failed totalization or unresolved descriptor bindings block an ALLOW decision. The contract

$$\text{Authorize}(\rho_d(u), P) \to \{\text{ALLOW}, \text{DENY}, \text{ABSTAIN}\}$$

receives the registered view and authenticated authority context P. Such a policy may check ACLs, delegation, visibility, quotas, or other resource-specific constraints. This equation specifies the interface whose input we validate. Registration supplies observability; P supplies authority.

The consumer can also receive a state-aware request: totalized arguments and selected trusted pre-state interpreted by a tool-specific adapter. The typed path places operation knowledge in a descriptor interpreter and declarative rules; the state-aware path places it in executable adapter code. The interface-economy audit measures their state exposure, partitioning, code structure, and latency.

Runtime instantiation reads totalized arguments, trusted provenance, and the current trusted pre-commit snapshot required by the descriptor. A state-derived atom is valid for that snapshot; freshness and check–use consistency must hold until the checked call commits.

Our end-to-end consumer uses resource ACLs, channel capabilities, and attenuated transfer delegations. Concrete post-state transitions and pre-commit atoms enter the same tri-state authority engine through their respective adapters. Tool name, raw arguments, common fields, and a state-aware request expose alternative interfaces; unknown policy facts produce ABSTAIN. Policy, authority, state, and decision logic remain fixed.

4.4 Integration Case Study

The AgentDojo case study attaches a frozen registered-field projection to the pre-commit hook; we call this configuration Provenance Monitor. The adapter totalizes fields, propagates marked provenance, and expands list values. Its provenance-origin policy rejects registered effects or values introduced solely by untrusted evidence outside the authenticated task; missing or unresolved bindings produce ABSTAIN. The executor commits the checked call and reconciles every decision with execution. Figure 4 details this path.

5 Security Analysis

5.1 Authorization Observability

For execution context u, let $\mu(u)$ be its committed effects and let $\gamma(u)$ contain policy-relevant metadata such as effect provenance. The ideal commit view is $\omega(u) = (\mu(u), \gamma(u))$. Each policy context $P \in \mathcal{P}$ induces an ideal decision $I_P(\omega(u))$. Our main experiment instantiates P with ACLs, capabilities, and delegations; a finite stress relation uses effect-set containment. The checked policy snapshot also governs the commit. A monitor sees $\rho(u)$ rather than $\omega(u)$; a descriptor instantiates $\rho_d(u) = A_{d_t}(u)$.

Representation induces the observational equivalence relation $u \sim_\rho v$ exactly when $\rho(u) = \rho(v)$. Every downstream authorizer whose call-dependent input is ρ is constant within each such class. The interface therefore upper-bounds the resolution of every downstream policy, independently of the policy language's sophistication.

Definition 1 (Authorization sufficiency)**.** A representation ρ is authorization-sufficient on domain D for policy family $\mathcal{P}$ when

$$\rho(u) = \rho(v) \Longrightarrow I_P(\omega(u)) = I_P(\omega(v))$$

for every $u, v \in D$ and $P \in \mathcal{P}$.

The definition applies to every representation. It permits distinctions ignored by $\mathcal{P}$ to remain merged and requires every policy-distinct pair to be separated.

Theorem 2 (Authorization-observability collision)**.** *If ρ is not authorization-sufficient, no deterministic monitor whose only call-dependent input is $\rho(u)$ can, even when given P, both*

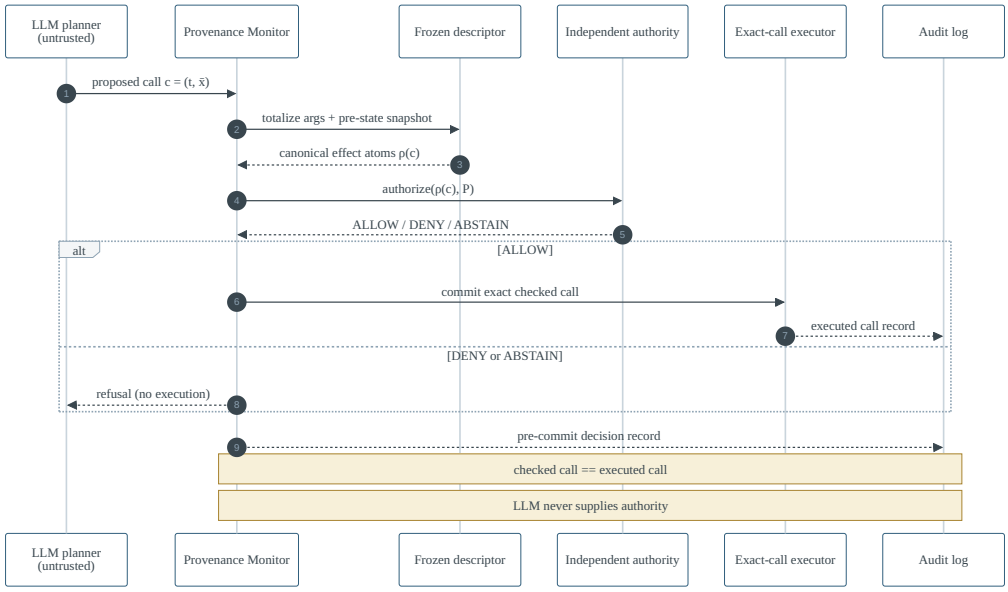

Figure 4: Online pre-commit consumption path. The planner only proposes calls; a deterministic monitor instantiates the frozen descriptor and consults independently supplied authority. The executor commits exactly the checked call, and both the decision and the execution are recorded for reconciliation.

allow every authorized context and reject every unauthorized context.

Proof sketch. Insufficiency gives u, v, P with equal representations and opposite ideal decisions. The monitor must return the same decision for both. Allowing admits the unauthorized member; denying or abstaining withholds the authorized member.

For fixed P, if a representation cell contains both an authorized and an unauthorized row, every sound monitor over that view must deny or abstain on the authorized row, yielding at least one false denial or abstention. This bound applies to tool-name, whole-call, and atom views alike.

This information bound applies equally to ACL, capability, provenance, and model-based policy consumers. Agent harnesses make it directly testable: schemas, defaults, aliases, and pre-state determine what a generated call commits, while authorization acts on the interface exposed before commitment. Validation exposes disagreements between these views as implementation-grounded counterexamples.

5.2 Executable Interface Falsification

An intervention $v = do_J(u)$ changes selected call fields, trusted provenance, or pre-state while holding the tool implementation fixed. Source execution observes committed state changes; the trusted harness records policy-relevant provenance and control metadata. Together they instantiate ω on the tested pair.

Proposition 3 (Counterfactual witness)**.** *If $\rho(u) = \rho(v)$ but some $P \in \mathcal{P}$ satisfies $I_P(\omega(u)) \neq I_P(\omega(v))$, then ρ is insufficient on every domain containing u and v.*

The proposition turns counterfactual testing into a falsification procedure. An output-only change with unchanged ω provides no witness. A trusted provenance change may provide one when policy distinguishes that provenance; sensitivity to a semantic-invariant placebo instead exposes overpartition.

An effect witness and a policy-separating witness form a failure certificate when the candidate representation merges the pair. Source execution, policy separation, and representation equality supply its three predicates.

For a finite domain, counterexample-guided refinement repeatedly splits a representation cell using a witnessed separating field or qualifier. Call a cell *policy-mixed* when it contains u, v for which at least one $P \in \mathcal{P}$ gives different ideal decisions.

Proposition 4 (Finite refinement termination)**.** *On finite D, a refinement procedure that never merges cells can perform at most $|D| - 1$ nonempty cell splits. If each step separates a witnessed authorization collision and the procedure stops only when no policy-mixed cell remains, the resulting representation is authorization-sufficient on $(D, \mathcal{P})$.*

Proof sketch. Every split strictly increases the number of nonempty cells, which is bounded by $|D|$. At termination, equal representations imply membership in the same non-

mixed cell. By the definition of policy-mixed, their ideal decisions agree for every $P \in \mathcal{P}$.

Generated interventions define D, and unresolved interventions remain in the validation record. Each accepted descriptor carries the $(D, \mathcal{P})$ pair on which the adequacy result applies.

5.3 Conditional Consumer Composition

The representation results become a commit guarantee only when the descriptor, policy, and enforcement path satisfy separate obligations. Let J_P be the ideal policy predicate expressed over a descriptor's atom view.

Proposition 5 (Conditional interface consumption)**.** *Assume four obligations for every reachable effectful call u. First, descriptor fidelity gives $J_P(A_{d_t}(u)) = I_P(\omega(u))$ for every admissible P. Second, the authorizer returns* ALLOW *only when $J_P(A_{d_t}(u)) = 1$. Third, every effectful call is mediated before commit. Finally, the executor commits exactly the checked call under the same policy snapshot. Then every allowed committed call satisfies $I_P(\omega(u)) = 1$.*

Proof sketch. Consider an allowed committed call u. Complete mediation supplies a check for u, and authorizer soundness gives $J_P(A_{d_t}(u)) = 1$. Descriptor fidelity therefore gives $I_P(\omega(u)) = 1$. Call and policy-snapshot check–use consistency make this conclusion apply at commitment.

Counterfactual tests probe descriptor fidelity. The explicit-authority experiment consumes the resulting interface through one ACL, capability, and delegation engine; AgentDojo instantiates a provenance-origin policy over registered fields.

5.4 Integration Instance: Provenance-Origin Confinement

For this integration policy, let $F(u)$ and $E(u)$ be the registered field values and effect kinds of call u. Let $U_V(h), U_E(h)$ denote values and requests from provenance-marked untrusted evidence, and let $G_V(q), G_E(q)$ denote those grounded in the authenticated task. The monitor allows only when

$$F(u) \cap U_V(h) \subseteq G_V(q) \quad \text{and} \quad E(u) \cap U_E(h) \subseteq G_E(q).$$

Corollary 6 (Conditional provenance-origin confinement)**.** *If the descriptor, provenance adapter, and grounding relation are sound for the registered fields and effect kinds, and every committed call is exactly the call checked, then an allowed call contains no registered effect or value introduced solely by untrusted evidence outside the authenticated task.*

Proof sketch. Any violating value lies in $F(u) \cap U_V(h)$ but outside $G_V(q)$, contradicting the first condition; the same argument over $E(u)$ contradicts the second for an effect-only violation. Exact mediation transfers the check to commitment.

This corollary characterizes the integration policy. The explicit-authority experiment evaluates ACL, capability, and delegation decisions separately; runtime audits here test mediation and check–use consistency. Appendix D gives the complete definitions and proof details.

6 Evaluation Methodology

The evaluation follows the security argument from implementation semantics to commit-time consumption. We first characterize effect distinctions, then use source-executed policy-separating pairs to falsify coarse interfaces and test frozen candidates. Holding calls, state, authority, and decision logic fixed isolates the authorization consequences of representation choice. Finally, an AgentDojo study exercises a registered interface at an audited pre-commit boundary.

Table 1 maps each evidence source to the claim it tests and the oracle that supplies its reference decision. Figure 5 visualizes the same chain.

Interpretation of evidence. The two result directions are asymmetric. One source-executed pair with different ideal decisions and the same representation conclusively falsifies that interface on any domain containing the pair. The absence of such a pair creates a conformance record for the implementation, interventions, and policy family used by the test. The artifact binds these elements and all unresolved tests to each accepted descriptor.

Three evidence sources have distinct roles. The *source execution oracle* answers what the implementation committed in a copied pre-state. The *policy-separation oracle* determines whether two observed semantic views require different decisions under the diagnostic policy family. The later *explicit-authority engine* is a concrete downstream consumer over ACLs, capabilities, and delegations. These components establish implementation behavior, policy relevance, and downstream consumption, respectively.

Implementation semantics. We replay all 339 calls in AgentDojo v1.1.2's 97 official benign trajectories from fresh prescribed states. Before/after differences are normalized into resource, target, visibility, message, membership, and external-interaction effects. We count effect occurrences and cross-type or cross-subsystem calls. A 56-call source domain adds payload, temporal, recurrence, repeated-effect, and state-dependent distinctions.

Interface falsification and post-freeze checking. The registration harness varies values, types, boundaries, list cardinality, omitted defaults, and pre-state under a fixed implementation; surface changes serve as invariance placebos. It classifies every attempt as a committed-effect change, output-only change, invariant, invalid, or unresolved. A failure certificate

Table 1: Evaluation evidence map. Each protocol supports one step of the security argument; evidence from a later stage does not retroactively certify an earlier one.

Stage	Domain	Purpose	Evidence / decision source	Supported claim
Source characterization	AgentDojo benign trajectories	Establish compound and state-dependent implementation behavior	Copied-sandbox before/after execution	Tool identity and call text may omit committed distinctions
Interface falsification	56-call source relation	Construct separating counterexamples	Source execution plus finite diagnostic policy relation	Coarse views are insufficient on witnessed pairs
Post-freeze check	11 tools, three public MCP implementations	Test a frozen candidate on descriptor-blind contexts	Independent source execution plus native-delta policy	Candidate preserves the exercised distinctions
Explicit authority	Four domains, 1,024 contexts	Measure downstream authorization consequences and a strong state-aware alternative	ACL, capability, and delegation engine	Sufficiency depends on preserved distinctions, not a privileged syntax
Integration	30 copied-sandbox trajectories; AgentDojo audit	Exercise pre-commit consumption and reconciliation	Explicit authority and native validators	A registered interface can be consumed at a check–use boundary

Figure 5: Evaluation evidence chain. Each stage supplies one step of the security argument; Table 1 gives the corresponding domains, oracles, and claims.

requires changed source behavior, a different policy decision, and an unchanged candidate representation.

The 56-call analysis uses a finite source-effect/submultiset relation for interface-discrimination stress, and a frozen initial set of 80 intervention executions drives refinement. The later authority protocol supplies deployment-style policies. For evidence from independently implemented tools, we pin a public filesystem server, SQLite server, and graph-memory server at specific commits. Their 11 stateful tools are selected outside our benchmark. Registration uses 12 counterfactual pairs per tool (24 registration contexts per tool, 264 in total). We then freeze the descriptors and execute 24 descriptor-blind contexts per tool with disjoint values, states, and list cardinalities. Each invocation runs the original MCP implementation in a fresh disposable fixture. To isolate construction from evaluation, descriptors are frozen before context generation, and the generator, source oracle, descriptor compiler, and native-policy path are implemented as separate modules. The native path derives decisions directly from raw filesystem, row, and graph deltas; the typed path compiles the pre-commit request for the authority engine. We compare tool name, raw call, state-aware request, common fields, typed effects, and a diagnostic source-effect oracle. Each validation record includes the implementation provenance.

Before evaluation outcomes are read, a mutation manifest deletes templates or pre-state dependencies, disables list expansion, removes qualifier bindings, swaps resource and target, collapses operations, or ignores no-op/default behavior when applicable. Every mutant is tested during registration and on the frozen descriptor-blind contexts. Mutants that change no decision in either frozen domain are recorded as decision-equivalent there.

Authorization under explicit authority. The downstream experiment uses four copied-sandbox domains: calendar, workspace sharing, messaging, and banking. Their authority state is expressed as resource ACLs, channel capabilities, and attenuated transfer delegations. The same tri-state engine consumes every representation. Calls, pre-states, authenticated subjects, authority state, and decision logic remain fixed; only the observation interface changes.

Registration executes 192 counterfactual pairs spanning resource, target, operation mode, qualifier, default, pre-state, and surface-invariant changes. The resulting typed-effect descriptors are hash-frozen and loaded as executable JSON. Evaluation uses 1,024 contexts materialized after freeze from a precommitted generator and authority configuration. The source transition oracle and descriptor compiler are separate implementations. The explicit-authority engine evaluates the source-observed transition to obtain the ideal decision. Each candidate representation is then compared against that decision to identify mixed representation cells.

We compare the tool-name view, canonical raw-call view, common-field view, and typed-effect interface. We additionally evaluate a state-aware request view containing totalized arguments and ACL-relevant attributes of directly referenced resources. Its collision track consists of this request view; a hash-frozen adapter supplies the disclosed tool-specific operation knowledge required by the shared authority engine. The raw-call view contains totalized call text, whereas the state-aware view also exposes trusted pre-state attributes. Both are evaluated under Definition 1.

We audit both consumption paths on the 1,024- and 264-context domains, recording representation cells, source-equivalent overpartition, serialized size, policy-facing trusted

state, code structure, descriptor size, and staged latency. Timing uses three warm-up and 30 complete measured batches.

Pre-commit integration. We first run 30 short trajectories over `schedule_meeting`, `share_document`, and `transfer_funds`. The local Qwen3-32B checkpoint proposes a structured call. The frozen descriptor instantiates effects, the explicit-authority engine decides, and an allowed checked call executes in the copied sandbox. Each commit is reconciled against both the checked call and observed post-state transition.

AgentDojo supplies 97 benign tasks and 629 official indirect-injection pairs. Under a fixed DeepSeek model (`deepseek-v4-flash`) and native validators, repeated runs compare no guard, Spotlighting [16], and the provenance-origin consumer from Section 5. A second common protocol runs no guard, Spotlighting [16], Prompt Sandwiching[1], a PromptArmor-style local adapter [32], and the consumer on the same Qwen3-32B checkpoint and exact 726 benchmark keys. Every decision is reconciled with the executed call. Public-family and AgentLAB saved-replay protocols test transfer, while a raw-field control measures whether typed semantic roles alter runtime decisions.

Metrics and integrity. Interface tests report mixed cells and policy-separating pairs. The authority experiment reports unsafe allow among ideal denials, authorized work withheld among ideal allows, coverage, abstention, exact-decision rate, and the unavoidable-error lower bound induced by mixed cells. A denial and an abstention are distinct monitor outcomes, but either withholds an authorized operation; coverage records how often the monitor commits to ALLOW or DENY. Native benchmark validators compute runtime outcomes. All protocols retain invalid and failed rows, run in copied or replayed environments, and produce no external side effects. Appendix B describes the fail-fast reproduction checks.

7 Results

7.1 Compound and State-Dependent Tool Effects

Tool calls can commit more than one independently relevant state change. Source replay observes a state or external-interaction effect in 29.5% of 339 AgentDojo calls. Among effectful calls, 17.0% contain multiple logical effect units and 13.0% cross effect types or benchmark subsystems (Table 2). In the ToolSandbox protocol [21], 9 of 32 contexts (28.1%) keep the tool and explicit arguments fixed while pre-state changes the committed effect. These execution facts establish the semantic distinctions; the separating relations below determine which of them matter to authorization.

[1]We implement the sandwich-defense recipe documented at https://learnprompting.org/docs/prompt_hacking/defensive_measures/sandwich_defense; it is a prompting technique rather than a single citable paper.

Table 2: Source-executed effects in AgentDojo v1.1.2 benign trajectories ($N = 339$ calls). Rates after the first row use the 100 effectful calls as the denominator.

Measure	Rate
Effectful calls	29.5%
Compound effects	17.0%
Heterogeneous effects	13.0%
Multiple targets	7.0%
Cross-subsystem effects	13.0%

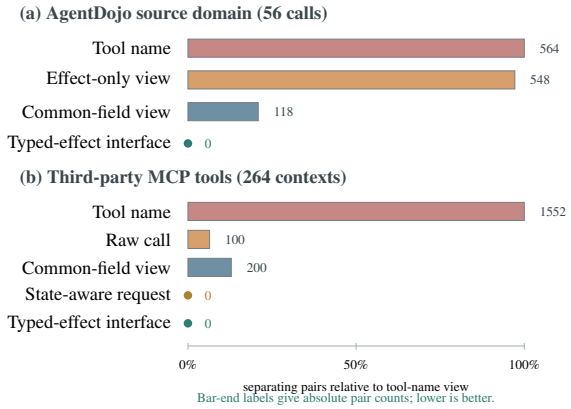

Figure 6: Constructive falsification and post-freeze interface conformance. Bars normalize policy-separating pair counts to the tool-name view within each domain; labels report absolute counts. Every nonzero count is a constructive counterexample to that view. Zero for the typed-effect interface records collision freedom for the enumerated domain and policy family.

7.2 Constructive Falsification and Interface Conformance

Executable failure certificates. Coarse views merge executions that the diagnostic policy relation separates. In the 56-call source domain, the tool-name and effect-only views induce 564 and 548 separating pairs. The common-field view leaves 118 pairs by merging payload, temporal, recurrence, and repeated-effect distinctions. Each pair is constructive: the tool executed, the committed semantic views differ, the policy-separation oracle requires different decisions, and the tested interface represents both executions identically. Any one pair therefore falsifies that interface on a domain containing it (Figure 6(a)).

The deterministic extractor reconstructs certificates from pre-state, calls, source transitions, decisions, and representation keys, selecting the first valid row in each of six frozen

Table 3: Two of the six executable certificates extracted by frozen category predicates and lexicographic selection.

Cause	Colliding view	Source-observed distinction
Compound effect	Raw call	Same transfer call; compliance pre-state adds an unauthorized override
State dependence	Raw call	Same transfer call; alias pre-state changes account and payee

Table 4: Post-freeze validation on 264 contexts from 11 tools in three pinned public MCP implementations. A mixed cell contains both authorized and unauthorized source executions.

Interface	Cells	Mixed	Separating pairs
Tool name	11	11	1,552
Raw call	155	5	100
State-aware request	161	0	0
Common fields	126	12	200
Typed effects	138	0	0

categories. Table 3 shows two of the six extracted certificates, both with identical call text: a compliance flag adds a compound banking effect, while an alias map redirects both account and payee. Each row rechecks source difference, policy separation, coarse equality, and refined distinction.

Counterexamples guide monotone refinement. Across 211 comparable edges in the 32-representation lattice, restoring a witnessed distinction never coarsens the partition or increases its collision lower bound. The inspected monotone refinement path adds date, subject, recurrence, payload, and visibility distinctions and reduces the observed separating-pair count from 124 to zero. This behavior instantiates Proposition 4 on the diagnostic relation.

Independent implementations after freeze. The three pinned MCP implementations execute without source failure in all 264 registration and 264 post-freeze contexts. A native-delta policy that consumes raw filesystem, row, and graph changes agrees with the typed path on every registration and post-freeze decision, with no typed compile failure. Tool name, raw call, and common fields retain 11, 5, and 12 mixed cells, respectively (Table 4). The typed interface and the stronger state-aware request view retain none.

A collision in a shipped interface. The public filesystem server's official schema describes `write_file` as one operation that can create or completely overwrite a file, but the tool-call interface it exposes to policy carries only the tool name and schema-validated arguments. In the frozen post-freeze contexts, two executions with identical call text commit different operations: one creates `team/eval-1-file.txt` and is allowed, while the other overwrites an existing file and is denied. This collision is therefore grounded in a shipped third-party interface rather than a synthetic view (Table 5).

Table 5: A concrete collision in the interface shipped by a public MCP filesystem server. Both executions use the same official tool name and schema-validated arguments; the official schema describes one operation that can create or completely overwrite a file. Pre-state changes the committed operation, and the policy family must decide the two executions differently.

Pre-state	Committed operation	Ideal decision
`write_file(path="team/eval-1-file.txt", content="approved update")`		
File absent	`filesystem.file.create`	ALLOW
File exists	`filesystem.file.overwrite`	DENY

Official schema description: "Create a new file or completely overwrite an existing file with new content." The shipped tool-call interface exposes only the tool name and arguments, so both executions share one representation key.

Table 6: Authorization outcomes under one shared explicit-authority engine (percent except mixed cells). "Unsafe" is normalized by ideal denials; "Withheld" counts ideal allows answered DENY or ABSTAIN; "Exact" requires the ideal ALLOW/DENY decision.

Interface	Unsafe	Withheld	Cov.	Exact	Mixed
Tool-name view	0.0	100.0	0.0	0.0	4
Raw-call view	1.1	50.4	99.6	87.2	51
Common-field view	0.0	32.9	81.0	81.0	9
State-aware request†	0.0	0.0	100.0	100.0	0
Typed-effect interface	0.0	0.0	100.0	100.0	0

† Direct metrics use a disclosed tool-specific request adapter; collision metrics use only the pure state-aware observation view.

Of 47 descriptor mutants generated without reading outcomes, registration detects 37. The remaining 10 are decision-equivalent on both frozen domains, and none first fails after freeze.

Unresolved evidence is retained. AgentDojo-wide registration keeps all 67 schema fields because invalid or unresolved interventions provide no evidence for omission. The result is a conservative full-field registry. Appendix C reports the field-level witness and failure counts.

7.3 Authorization under Explicit Authority

The explicit-authority protocol tests the validation framework rather than a single representation. Two independently constructed interfaces preserve every distinction it exercises. The typed-effect descriptors match the transition oracle on all 384 registration contexts and satisfy all 192 counterfactual relation checks. Across 1,024 contexts materialized after descriptor freeze, no policy-mixed cell is observed, and the shared authority engine reproduces every source-derived decision (Table 6).

Changing only the observation interface exposes the cost of erased distinctions. The tool-name view abstains on every context and therefore withholds all 240 authorized operations. The raw-call view covers 99.6% of contexts but unsafely allows 1.1% of ideal denials and withholds 50.4% of authorized

operations. The common-field view makes no incorrect covered decision, but abstention reduces coverage to 81.0% and withholds 32.9% of authorized operations. Its nine mixed cells also show that no deterministic consumer over that view can be both sound and fully permissive on this domain.

The stronger state-aware request view also has no mixed cell and supports exact decisions on all 1,024 contexts. Its pure representation contains only totalized arguments and trusted attributes of directly referenced resources; the authorization track discloses a static tool-specific operation adapter. It is larger and more partitioned than the typed interface (669 versus 297 cells, with 3,252 source-equivalent pairs split), but it is sufficient on this domain. The experiment therefore identifies two sufficient interfaces with different semantic placement.

What normalization buys. Both paths reproduce all decisions, with different policy-facing interfaces. The typed interface is less fragmented: state-aware requests split 3,252 and 90 source-equivalent pairs in the 1,024- and 264-context domains, whereas the typed view splits none. Their median policy-facing views contain 9 tool-pre-state leaves in the 1,024-context domain and 4 in the 264-context domain, versus none after typed instantiation. The implementations expose different cost profiles. In the four-tool prototype, the generic typed interpreter is about 5.2× slower than the state-aware path. In the third-party prototype it is about 1.4× faster, and its hard-coded compiler is larger than the state-aware adapter. Across both domains, the typed interface consistently provides canonicalization, lower overpartition, and descriptor-local semantic adaptation.

Pre-state explains part of the raw-call gap. Among 52 groups with identical arguments but state-dependent ideal decisions, call text collides in every group; the typed-effect and state-aware interfaces separate all of them. Explicit call text alone is therefore insufficient in these contexts; adding trusted state can recover the distinction.

7.4 Pre-Commit Consumption

In the 30-trajectory copied-sandbox study, the local Qwen3-32B checkpoint produces a parseable structured call for every task. The explicit-authority consumer allows all 10 authorized or within-delegation calls and denies all 20 calls with an unauthorized target, compound unauthorized effect, adverse pre-state, or exceeded delegation. All 10 committed calls equal the checked call and their source-observed transitions reconcile with the instantiated atoms. The scenario-balanced study therefore exercises the complete call–descriptor–atom–authority–execute path.

The AgentDojo registered-field projection drives an audited pre-commit hook. In the official DeepSeek run, Provenance Monitor allows 2,693 of 2,783 checks (96.8%), and denies 90, with no abstention; it produces no unreconciled execution and performs authorization without a runtime LLM call. Missing or unresolved bindings would yield ABSTAIN. In the matched Qwen3-32B run, Provenance Monitor reduces attack success relative to no guard while benign utility remains close; Prompt Sandwiching and the PromptArmor-style adapter provide stronger security–utility points, so runtime numbers exercise integration rather than dominate prompting defenses. Appendix C reports complete DeepSeek, Qwen3-32B, held-out, and transfer results.

8 Limitations

Coverage and semantic completeness. The validation record covers the frozen implementation, interventions, and policy family. Unseen defaults, hidden effects, asynchronous work, general aliases, and untested states may require additional distinctions. Invalid or unresolved interventions lead to conservative retention. The AgentDojo registry therefore retains all 67 schema fields. Counterexamples guide refinement within the recorded domain; globally complete descriptor synthesis remains open.

Semantic independence. The MCP implementations have independent public authorship, whereas the descriptors, policies, and both decision paths were constructed in this project. Native deltas, import isolation, descriptor-blind context generation, and systematic mutations reduce shared-helper circularity. Independent descriptor and policy authors would provide stronger semantic certification.

State and execution assumptions. Registration binds evidence to an implementation identity. Remote services and evolving APIs require versioning and renewed validation when their semantics change. State-dependent instantiation further requires a fresh trusted snapshot and check–use consistency. The prototype provides neither general concurrency control nor transactional protection against TOCTOU races.

Interface economy. The validation method is representation-agnostic: the state-aware request is also sufficient on both evaluated domains, and typed effects are one evaluated candidate rather than a claimed optimum. Typed effects reduce source-equivalent overpartition and policy-facing raw-state exposure in these prototypes. Latency and executable code size vary by implementation, so the audit establishes no general TCB, extensibility, or performance advantage.

Harness integration. DeepSeek Harness demonstrates an industrial pre-execution pipeline whose guards observe tool calls and approve or deny them before execution. Our method targets the observation object consumed by such pipelines:

mounting a frozen descriptor as one of its monotonic guards would preserve the same check–use boundary, but this port and its evaluation remain future work.

Runtime consumer. The AgentDojo integration enforces provenance-origin confinement over a conservative registered-field projection. It does not implement general organizational ACLs, delegation, quotas, or output-only goals. Prompt Sandwiching provides a better security–utility point in the matched Qwen3-32B comparison. The repeated DeepSeek run also misses the pre-registered benign non-inferiority margin by 0.4 percentage points, and saved AgentLAB transfer loses 17.5 percentage points of utility. The comparable prompting adapters follow this paper's common-input protocol; their results are not reproductions of every original-paper evaluation.

9 Conclusion

Authorization logic is only as discriminating as the interface it observes. When an interface merges executions that policy must decide differently, no downstream ACL, capability system, provenance check, or model can recover the lost distinction. A source-executed policy-separating collision is therefore a concrete failure certificate for that interface.

Our method turns this reference-monitor precondition into an executable security contract. Controlled interventions establish what a tool commits, a policy-separation oracle identifies the distinctions that matter, and representation collisions expose inadequate interfaces before deployment. Counterexamples drive refinement, while the frozen validation record binds an accepted descriptor to its implementation and evidence.

We instantiate this discipline through typed effects and compare it with a strong state-aware request interface; both suffice in the evaluated domains, so the method validates representations rather than promoting one. The broader result is a practical rule for agent harnesses: test the authorization interface against implementation behavior before downstream policy trusts it.

References

[1] Hengyu An, Jinghuai Zhang, Tianyu Du, Chunyi Zhou, Qingming Li, Tao Lin, and Shouling Ji. Ipiguard: A novel tool dependency graph-based defense against indirect prompt injection in llm agents, 2025. EMNLP 2025.

[2] Rahul Suresh Babu and Laxmipriya Ganesh Iyer. Contract2Tool: Learning preconditions and effects for reliable tool-augmented LLM agents, 2026.

[3] Sander Beckers and Joseph Y. Halpern. Abstracting causal models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 2678–2685, 2019.

[4] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory – ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330. Springer, 2001.

[5] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009.

[6] Edmund M. Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. In *Computer Aided Verification*, volume 1855 of *Lecture Notes in Computer Science*, pages 154–169. Springer, 2000.

[7] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramer. Defeating prompt injections by design, 2025.

[8] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramer. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, 2024.

[9] DeepSeek-AI. Deepseek harness, 2026. Developer preview.

[10] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976.

[11] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.

[12] Linfeng Fan, Ziwei Li, Yuan Tian, Yichen Wang, Rongsheng Li, and Xiong Wang. The granularity mismatch in agent security: Argument-level provenance solves enforcement and isolates the LLM reasoning bottleneck, 2026.

[13] Atticus Geiger, Zhengxuan Wu, Hanson Lu, Josh Rozner, Elisa Kreiss, Thomas Icard, Noah D. Goodman, and Christopher Potts. Inducing causal structure for interpretable neural networks. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 7324–7338. PMLR, 2022.

[14] Norm Hardy. The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.

[15] Yu He, Haozhe Zhu, Yiming Li, Shuo Shao, Hongwei Yao, Zhihao Liu, and Zhan Qin. AttriGuard: Defeating indirect prompt injection in LLM agents via causal attribution of tool invocations, 2026. Accepted by USENIX Security 2026.

[16] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. Defending against indirect prompt injection attacks with spotlighting, 2024.

[17] Yue Huang, Hang Hua, Yujun Zhou, Pengcheng Jing, Manish Nagireddy, Inkit Padhi, Greta Dolcetti, Zhangchen Xu, Subhajit Chaudhury, Ambrish Rawat, Liubov Nedoshivina, Pin-Yu Chen, Prasanna Sattigeri, and Xiangliang Zhang. Building a foundational guardrail for general agentic systems via synthetic data, 2025.

[18] Laxmipriya Ganesh Iyer and Rahul Suresh Babu. The gate is only as honest as its contracts: ContractGuard for the contract layer of risk-aware causal gating, 2026.

[19] Tanqiu Jiang, Yuhui Wang, Jiacheng Liang, and Ting Wang. AgentLAB: Benchmarking LLM agents against long-horizon attacks, 2026.

[20] Minbeom Kim, Mihir Parmar, Phillip Wallis, Lesly Miculicich, Kyomin Jung, Krishnamurthy Dj Dvijotham, Long T. Le, and Tomas Pfister. CausalArmor: Efficient indirect prompt injection guardrails via causal attribution, 2026.

[21] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1160–1183, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics.

[22] John M. Lucassen and David K. Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 47–57. Association for Computing Machinery, 1988.

[23] Jiaqi Luo, Songyang Peng, Jiarun Dai, Zhile Chen, Zhuoxiang Shen, Geng Hong, Xudong Pan, Yuan Zhang, and Min Yang. AgentGuard: An attribute-based access control framework for tool-use LLM-based agent, 2026.

[24] Yuhan Ma and Stefan Schmid. SecureClaw: Clawing back control of LLM agents, 2026.

[25] Yutao Mou, Zhangchi Xue, Lijun Li, Peiyang Liu, Shikun Zhang, Wei Ye, and Jing Shao. Toolsafe: Enhancing tool invocation safety of llm-based agents via proactive step-level guardrail and feedback, 2026. Work in progress.

[26] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2 edition, 2009.

[27] Arun Ravindran and Saurabh Deochake. ToolGuardian: Declarative security for AI agent-tool interactions, 2026.

[28] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox, 2023.

[29] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[30] Igor Santos-Grueiro. Temporary authority, permanent effects: Commit-time authorization for LLM agents, 2026.

[31] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing ai agents with privilege control, 2025.

[32] Tianneng Shi, Kaijie Zhu, Zhun Wang, Yuqi Jia, Will Cai, Weida Liang, Haonan Wang, Hend Alzahrani, Joshua Lu, Kenji Kawaguchi, Basel Alomair, Xuandong Zhao, William Yang Wang, Neil Gong, Wenbo Guo, and Dawn Song. Promptarmor: Simple yet effective prompt injection defenses, 2025.

[33] Yifan Shi, Wei Zhang, and Tianyi Cui. A programming paradigm for spatiotemporal composability, 2026. Preprint draft of August 13, 2026.

[34] Vincent Siu, Jingxuan He, Kyle Montgomery, Zhun Wang, Neil Gong, Chenguang Wang, and Dawn Song. A framework for formalizing LLM agent security, 2026.

[35] Huiri Tan, Yikun Wang, Puyang Zhang, Shangyu Li, and Jiasi Shen. ETAS: An effect-typed language for agent systems, 2026.

[36] Peiran Wang, Ying Li, and Yuan Tian. Aligning provenance with authorization: A dual-graph defense for LLM agents, 2026.

[37] Wei Zhao, Zhe Li, Peixin Zhang, and Jun Sun. ClawGuard: A runtime security framework for tool-augmented LLM agents against indirect prompt injection, 2026.

[38] Jinhao Zhu, Kevin Tseng, Gil Vernik, Xiao Huang, Shishir G. Patil, Vivian Fang, and Raluca Ada Popa. Miniscope: A least privilege framework for authorizing tool calling agents, 2025.

[39] David Mellafe Zuvic. Capability gates are not authorization: Confused-deputy failures in LLM agent frameworks, 2026.

A Ethical Considerations

All tool executions occur in public benchmark sandboxes or copied local state. No experiment sends a message, transfers money, changes a cloud resource, or commits another real external effect. API models receive benchmark task text and synthetic benchmark content, not private user data. Credentials are provided only through environment variables and are excluded from logs and artifacts. Released adversarial material is limited to public benchmark families and the fixed evaluation records needed to reproduce defensive claims. Incorrect descriptors or misplaced confidence could create harm, so the artifact retains unresolved registration cases, failed interventions, and negative utility results alongside accepted descriptors.

B Open Science

The artifact package records source-executed effect oracles, frozen intervention and case manifests, conservative descriptors, row-level benchmark outputs, official-validator results, model identifiers and hashes, runtime decisions, tests, and environment locks. A fail-fast status manifest identifies the frozen artifacts required by each reported result; the release package includes only claims whose required artifacts and keys pass that check. Sidecar labels and source effects are used only for scoring and do not enter model prompts or deployable inputs. Before release, the package is rebuilt from the final manuscript state and scanned to exclude credentials, absolute paths, repository history, and identifying metadata.

Protocols hash model files, descriptors, case manifests, registered source, and monitor code. Finalizers reject duplicate or missing keys, execution errors, method-specific denominators, changed sources, and mismatches between pre-commit records and executed calls. Pre-committed evaluation cases use a hash ordering recorded in the artifact.

For the explicit-authority experiment, the artifact additionally includes the authority state, generator configuration, registration contexts, executable descriptor hashes, post-freeze evaluation contexts, source transitions, and per-representation decisions. The transition oracle and descriptor compiler are separate modules, and the reproduction gate checks all reported rows and table values.

The third-party validation freezes public repository commits, licenses, MCP schemas, implementation hashes, 264 registration executions, and 264 disjoint post-freeze contexts. Each call runs in a disposable filesystem, database, or memory fixture. The evaluation generator is descriptor-blind, and zero human refinements are recorded. Executable failure certificates are selected by frozen category predicates rather than favorable manual choice. The small runtime study records model output, totalized call, descriptor and authority hashes, atoms, decision, checked and executed calls, post-state, and transition reconciliation for each trajectory.

Table 7: Representation collisions over 56 source-executed AgentDojo calls. "Pairs" counts authorization-separating pairs under the finite submultiset policy family.

Interface	Mixed	Pairs	Finite suff.
Tool name	5	564	no
Effect-only view	6	548	no
Common-field view	5	118	no
Typed-effect interface	0	0	yes
Full source effect	0	0	yes

Table 8: Pre-registered, separate-source check on 32 contexts for five public ToolSandbox tools. "Coll." counts interface cells containing distinct source-observed effect sets; "Sep." counts authorization-separating pairs. All eight expected invalid call/state contexts remain in the denominator; the descriptor is not revised after outcomes become visible.

Interface	Cells	Coll.	Sep.	Overpart.
Tool name	5	5	88	16
Common-field view	11	4	41	0
Typed-effect interface	25	0	0	0
Source effect	25	0	0	0

C Additional Evaluation Detail

Exact representation diagnostics. Figure 6 visualizes the separating-pair lower bound. Tables 7 and 8 retain the exact cell, collision, and overpartition diagnostics.

Finite containment diagnostic. Before the explicit-authority benchmark, we tested whether each representation could reproduce a finite source-effect containment relation over 232 ordered ToolSandbox context pairs. Concrete typed atoms match that oracle with 0.0% UPA and 0.0% false denial; the relation is a representation stress test rather than a deployment authorization model (Table 9).

Exact DeepSeek runtime results. Table 10 retains the same-model values and the scope-aligned comparison summarized by Figure 7.

Runtime attribution and transfer. Table 11 reports the fixed applicability subset and saved AgentLAB transfer under their frozen protocols.

Table 9: Finite ToolSandbox authorizer check. UPA is unsafe pre-allow; FD is false denial.

View	UPA	FD	Coverage	Accuracy
Tool name	100.0%	0.0%	100.0%	35.3%
Exact raw args	9.3%	34.1%	100.0%	81.9%
Common-field view	56.7%	0.0%	100.0%	63.4%
Concrete atoms	0.0%	0.0%	100.0%	100.0%
Source oracle	0.0%	0.0%	100.0%	100.0%

All 232 ordered queries use one executed context as the allowed effect multiset and check another context of the same tool.

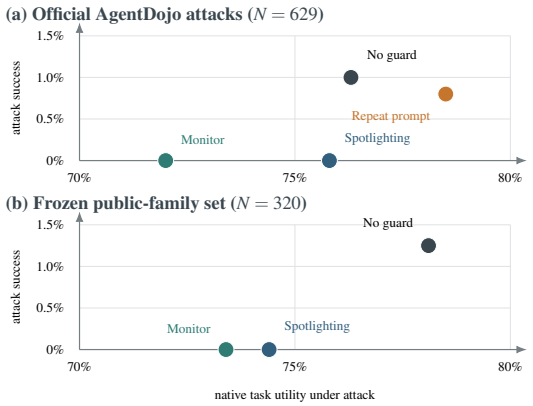

Figure 7: Bounded runtime characterization under one DeepSeek model. Lower attack success and higher task utility are preferable. The monitor removes the observed injection successes in both protocols, but does not dominate the prompting baseline on utility. Panels use separate frozen case manifests and must be interpreted within, not across, protocols; these results do not prove authorization-interface conformance.

Matched runtime comparisons. Table 12 reports the complete matched comparison. Across four interleaved DeepSeek repetitions, benign utility is 77.6% without a guard, 78.6% under Spotlighting, and 76.5% under Provenance Monitor. The mean monitor-minus-no-guard difference is -1.0 percentage points. The task-clustered one-sided 95% lower bound is -5.4 percentage points, below the pre-registered -5.0-point margin; the run therefore misses the non-inferiority criterion by 0.4 percentage points.

The matched Qwen3-32B run retains every exact benchmark key for all five methods. No guard, Spotlighting, Prompt Sandwiching, the PromptArmor-style adapter, and Provenance Monitor respectively achieve benign utility 61.9%, 66.0%, 68.0%, 27.8%, and 60.8%; their attack success is 10.0%, 9.5%, 1.0%, 0.0%, and 2.9% ($N = 97$ benign and $N = 629$ attack keys per method). Prompt Sandwiching provides the best combined utility–security point in this comparison. The PromptArmor-style adapter eliminates observed attack success while sharply reducing both utility measures.

On the frozen 320-case public-family set, attack success is 1.2% without a guard, 0.0% under Spotlighting, and 0.0% under Provenance Monitor; native task utility is reported for the same rows in Table 13.

On the separately frozen 40-key worst-of-four public-family set, attack success is 10.0% without a guard and 5.0% under the monitor; terminal utility is 45.0% and 42.5%.

Table 10: Same-model DeepSeek AgentDojo attack comparison ($N = 629$). AU is utility under attack. The scope-aligned comparison uses $N = 608$ after excluding one disclosed repair case and 20 output-only goals.

Method	AU	Attack success
No guard	76.3%	1.0%
Repeat user prompt	78.5%	0.8%
Spotlighting	75.8%	0.0%
Provenance Monitor	72.0%	0.0%

Scope aligned: no guard 1.0%; Provenance Monitor 0.0% ($p = .0156$).

Table 11: Closed-loop monitor-view comparison and fixed saved-attack transfer. Panel (a) uses 48 benign and 273 attack cases from a selection-conditioned AgentDojo subset. Panel (b) replays 303 fixed AgentLAB task-injection artifacts saved under AgentDojo v1.2.1, while the main AgentDojo results use v1.1.2; it is not adaptive attack generation.

Setting	Benign utility	Attack utility	ASR
(a) Monitor representation, AgentDojo v1.1.2			
No blocking	62.5%	56.0%	13.9%
Whole-call provenance	81.3%	21.2%	4.0%
Effect only	77.1%	53.8%	7.7%
Raw-field taint	81.3%	59.3%	3.7%
Registered fields	64.6%	54.2%	2.6%
(b) Fixed AgentLAB transfer, AgentDojo v1.2.1			
No defense	–	61.1%	31.4%
Provenance-origin monitor	–	43.6%	0.3%

Controlled monitor stresses. Table 14 shows that released checkpoints respond to some effect, resource, and authorization changes, but none binds every tested axis jointly. The hard guard's UPA rises from 3.6% on the unified suite to 38.3% on the repaired resource/authorization stress. Removing resource or authorization matching raises UPA to 61.7% and 89.2%, respectively. These are adapted custom stresses rather than the original systems' benchmark protocols.

Registration failures. Table 15 summarizes the multi-base AgentDojo audit, which attempts 1,325 source executions over 75 suite-scoped field instances. Of these, 76.8% are valid, 73.3% change committed state, 0.8% change only the returned output, 2.7% are effect-invariant, and 23.2% are invalid or unresolved. Most invalid interventions substitute nonexistent resource identifiers or violate a tool precondition. We retain those fields in the descriptor; absence of a valid witness is never interpreted as irrelevance. The artifact includes one row per attempted execution and a per-field summary.

Table 12: Matched runtime comparison. DeepSeek benign utility uses four interleaved repetitions ($N = 388$ per method). Qwen3-32B uses the same checkpoint and exact 97 benign and 629 attack keys for every method. Spotlighting denotes AgentDojo's built-in paper-defined defense; daggered methods are comparable local adapters, not original-paper reproductions.

Model / evaluation	Method	Benign utility	Attack utility	Attack success	Runs
DeepSeek benign	No guard	77.6%	–	–	4
DeepSeek benign	Spotlighting	78.6%	–	–	4
DeepSeek benign	Provenance Monitor	76.5%	–	–	4
Qwen3-32B	No guard	61.9%	51.5%	10.0%	1
Qwen3-32B	Spotlighting	66.0%	54.8%	9.5%	1
Qwen3-32B	Prompt Sandwiching†	68.0%	59.5%	1.0%	1
Qwen3-32B	PromptArmor-style†	27.8%	24.2%	0.0%	1
Qwen3-32B	Provenance Monitor	60.8%	50.6%	2.9%	1

†Comparable local prompting adapters implemented for this paper under the common-input protocol, not reproductions of any original-paper evaluation.

Table 13: Frozen public-family result under one DeepSeek protocol ($N = 320$ per method).

Method	Utility	Attack success
No guard	78.1%	1.2%
Spotlighting	74.4%	0.0%
Provenance Monitor	73.4%	0.0%

Public-family sensitivity. The current diagnostic uses a fixed set of 40 official attack keys and crosses each with four public AgentDojo attack generators. Worst-of-four attack success is 10.0% without defense and 5.0% under the provenance-origin monitor; terminal utility is 45.0% and 42.5%. The four attack-discordant pairs give two-sided exact McNemar $p = .625$.

Cost accounting. Table 16 separates interface normalization from prototype implementation cost. Typed views are less overpartitioned and expose no raw tool pre-state at the policy-facing boundary in these two designs, but neither latency nor executable code size consistently favors them.

The deterministic comparison kernel has sub-millisecond median latency in the reported microbenchmarks. This excludes descriptor registration, model inference, sandbox execution, and I/O. Historical complete-trajectory runs have median duration 53.27 s without mediation and 85.54 s under an earlier mediation protocol, but they were sequential and used different recovery paths, so we report the difference descriptively. The current provenance-origin guard makes no runtime LLM call; offline registration bears all proposal and counterfactual-execution cost.

Table 14: Controlled counterfactual problem characterization. Panel (a) reports group-level sensitivity to effect, authorization, and resource changes; higher is better. Panel (b) reports row-level unsafe pre-allow (UPA), safe false denial (FD), and coverage. These adapted custom stresses are not original-paper benchmark reproductions.

(a) Partial binding across monitors

Method	Effect	Auth.	Resource	UPA
Tool-name proxy	0.0%	0.0%	0.0%	29.2%
Qwen self-audit	66.7%	56.5%	29.2%	0.4%
TS-Guard	62.5%	76.4%	83.3%	15.9%
Safiron	100.0%	8.8%	16.7%	54.5%

(b) Hard-guard granularity stress

Guard / stress	Rows	UPA	FD	Coverage
Unified suite	822	3.6%	6.5%	91.7%
Resource/auth.	240	38.3%	0.0%	92.1%
w/o resource	240	61.7%	0.0%	93.8%
w/o authorization	240	89.2%	0.0%	92.1%
Provenance	336	0.0%	18.8%	54.5%

D Formal Definitions and Proof Details

Decision model. Let D be a finite set of reachable execution contexts and $\mathcal{P}$ the admissible per-commit policy contexts. For $u \in D$, $\Delta(u)$ is the finite ordered trace of security-relevant concrete effect occurrences, $\mu(u)$ is its induced multiset, and $\gamma(u)$ is the policy-relevant execution metadata associated with the commit. The ideal semantic view is $\omega(u) = (\mu(u), \gamma(u))$. Each $P \in \mathcal{P}$ induces the ideal decision $I_P(\omega(u))$; multiset containment is the instance $I_B(\omega(u)) = [\mu(u) \preceq B]$. The snapshot P used by the check is also the snapshot governing the corresponding commit. A deterministic monitor over representation ρ is

$$M : \mathrm{range}(\rho) \times \mathcal{P} \to \{\textsc{Allow}, \textsc{Deny}, \textsc{Abstain}\}.$$

It is sound on D when $M(\rho(u), P) = \textsc{Allow} \Rightarrow I_P(\omega(u)) = 1$. It is permissive when $I_P(\omega(u)) = 1 \Rightarrow M(\rho(u), P) = \textsc{Allow}$. Abstention may preserve safety but does not commit authorized work, so it is a loss of permissiveness in this statement.

Table 15: Multi-base source-executed registration audit over 75 suite-scoped field instances and 1,325 intervention executions. "Witness" means that a valid mutation changed committed state.

Registration outcome	Rate
Unique fields retained	100.0%
Fields with ≥ 5 valid intervention kinds	50.7%
Fields with a committed-effect witness	88.0%
Valid intervention executions	76.8%
Committed-effect-changing executions	73.3%
Output-only executions	0.8%
Effect-invariant executions	2.7%
Invalid or unresolved executions	23.2%

Proof of Theorem 2. Because ρ is not authorization-sufficient, there are $u, v \in D$ and $P \in \mathcal{P}$ with $\rho(u) = \rho(v)$ and different ideal decisions. Rename the pair so that $I_P(\omega(u)) = 1$ and $I_P(\omega(v)) = 0$. Since the monitor views are identical, determinism gives

$$M(\rho(u), P) = M(\rho(v), P) = d.$$

If $d = \textsc{Allow}$, soundness fails on v. If $d \in \{\textsc{Deny}, \textsc{Abstain}\}$, permissiveness fails on u. Every possible decision violates at least one property, proving the theorem. □

Randomized monitors. The same obstruction applies to probability-one guarantees. Equal inputs induce the same output distribution. Assigning positive probability to ALLOW gives positive unsafe-allow probability on the unauthorized context; assigning probability one to non-allow decisions gives false denial or abstention with probability one on the authorized context. Randomization can trade the two errors but cannot eliminate both.

Proof of Proposition 3. The separating pair directly violates Definition 1. Theorem 2 then gives the downstream monitor consequence. □

Proof of Proposition 4. Suppose the initial representation has $c_0 \geq 1$ nonempty cells. Each successful counterexample-guided step splits at least one nonempty cell into two nonempty cells, and no later step merges cells. After k refinements there are at least $c_0 + k$ cells, while a partition of D has at most $|D|$ cells. There can therefore be at most $|D| - c_0 \leq |D| - 1$ such splits. If the stopping condition leaves no policy-mixed cell, then for every pair in that cell and every $P \in \mathcal{P}$ the ideal decisions agree. Equality of representations therefore implies equality of the full policy-decision vector, which is exactly authorization sufficiency on $(D, \mathcal{P})$. □

Field necessity. Let π_{-f} erase atom component f. Component f is necessary on $(D, \mathcal{P})$ if $\pi_{-f} \circ \rho_K$ is not authorization-sufficient although ρ_K is. Theorem 2 then shows that erasing f forces an unsafe allow or loss of permissiveness for some admissible policy. This is a relative necessity result: changing D or the admissible policy family can change which components are necessary.

Proof of Proposition 5. Let u be any allowed committed effectful call. Complete mediation gives a pre-commit decision over $A_{d_t}(u)$, and the authorizer's allow rule gives $J_P(A_{d_t}(u)) = 1$. Descriptor fidelity yields $I_P(\omega(u)) = 1$. Check–use consistency ensures that the context whose semantic commit view is $\omega(u)$ is exactly the checked context and that the same policy snapshot governs its commitment. □

Proof of Corollary 6. Consider an allowed and committed call. Descriptor soundness puts each of its effect-bearing values in $F(u)$. Provenance soundness puts every value introduced by untrusted evidence in $U_V(h)$, and grounding soundness excludes it from $G_V(q)$ unless the authenticated task independently supports it. The guard's first allow condition therefore rejects a solely untrusted value. If an untrusted instruction changes only a registered effect kind, provenance soundness places that kind in $E(u) \cap U_E(h)$ and grounding soundness excludes it from $G_E(q)$; the second allow condition rejects it. Thus no allowed registered effect or effect-bearing value is introduced solely by untrusted evidence. Complete mediation applies this argument to every effectful commit, and check–use consistency ensures the argument concerns the call actually executed. □

Proof obligations. The observability result is parameterized by D and $\mathcal{P}$. The consumption result additionally assumes descriptor fidelity, policy correctness, policy-snapshot integrity, complete mediation, and check–use consistency. The provenance-origin corollary instantiates these obligations with the registered-field descriptor, provenance adapter, and grounding relation defined in Section 5. Operational limits of these assumptions are collected in Section 8.

Table 16: Prototype interface-economy audit. Both interfaces make exact decisions and have no mixed cell. "State leaves" counts tool pre-state values crossing the policy-facing boundary; authority state is excluded. Code and latency describe these implementations, not production TCB or performance.

Domain	Interface	Cells	Overpart.	Median bytes	State leaves	AST stmts	Median μs
Explicit authority (1,024)	Typed effect	297	0	241	0	132	73.0
	State-aware request	669	3,252	537	9	143	13.8
Third-party MCP (264)	Typed effect	138	0	203	0	102	10.0
	State-aware request	161	90	271	4	83	13.6